

Endpoint URL+Api 명세서

평가용 API 명세서

제10회 2026 미래에셋증권 AI Festival 공시 질의응답 과제의 평가용 API 서버 명세다.
주최 측 평가 시스템이 참가팀 서버로 질문을 보내고, 참가팀 서버가 답변을 돌려준다.

- 구현: 이 리포지토리의 `app.py` → `integration/api.py::create_app()`
- 응답 형식 변환: `integration/api.py::to_submission_response()`
- 상위 요구사항: `docs/contest-requirements.md` 3절
- 최종 수정: 2026-09-06

1. Endpoint URL

항목	값
제출용 Endpoint	<code>http://49.50.142.35/answer</code>
Base URL	<code>http://49.50.142.35</code>
프로토콜	HTTP/1.1 (TLS 없음)
포트	<code>80</code> (컨테이너 내부 8000으로 전달, NCP ACG 인바운드 <code>TCP 80</code> 허용)
서버	NCP, <code>root@49.50.142.35</code> , Docker Compose 프로젝트 <code>/root/dis-164</code> , 이미지 <code>dis164-agent:local</code>
프로세스	<code>uvicorn app:app --host 0.0.0.0 --port 8000 --workers 1</code>

평가 담당자는 이 서버에 직접 `GET` 요청을 보낸다.

노출 경로

메서드	경로	용도
<code>GET</code>	<code>/answer</code>	질문 1건 처리. 평가 대상.
<code>GET</code>	<code>/ready</code>	파이프라인 실행 준비 상태 확인.
<code>GET</code>	<code>/health</code>	프로세스 liveness 확인.
<code>GET</code>	<code>/openapi.json</code>	OpenAPI 3.1 스키마 (FastAPI 자동 생성).
<code>GET</code>	<code>/docs</code>	Swagger UI.
<code>GET</code>	<code>/redoc</code>	ReDoc UI.

2. 인증과 전송

- 인증 헤더 없음. 접근 제어는 NCP ACG의 IP 대역 제한으로만 한다.
- 요청 본문 없음. 모든 입력은 URL 쿼리 문자열로 전달한다.
- 요청 파라미터 값은 URL 인코딩한다(한글 질문 포함).
- 응답 `Content-Type` : `application/json`. 인코딩 UTF-8. 한글은 이스케이프하지 않는다.
- 상태 코드로 성공(200)과 실패(4xx / 5xx)를 구분한다.

3. GET /answer — 요청

3-1. 쿼리 파라미터

이름	타입	필수	설명
<code>question_id</code>	string	예	질문 식별자. 응답에 그대로 되돌려준다. 실행 중 불변.
<code>question</code>	string	예	자연어 질문 원문. 실행 중 불변.

두 파라미터는 모두 필수다. 하나라도 없으면 422를 반환한다(4-1 참고).

3-2. 요청 예시 — cURL

```
curl -G "<http://49.50.142.35/answer>" \
  --data-urlencode "question_id=Q-001" \
  --data-urlencode "question=삼성전자의 2024년 연결기준 영업이익은 얼마인가?"
```

3-3. 요청 예시 — Python

```
import requests

resp = requests.get(
    "<http://49.50.142.35/answer>",
    params={
        "question_id": "Q-001",
        "question": "삼성전자의 2024년 연결기준 영업이익은 얼마인가?",
    },
)
```

```

        timeout=310,
    )
    resp.raise_for_status()
    result = resp.json() # 4절 응답 스키마

```

4. GET /answer — 응답 (성공, 200 OK)

본문은 JSON 객체 하나다. 필드는 정확히 5개이며 모두 문자열이다.

(`scripts/smoke_api.py` 가 `set(response) == {5개 필드}` 와 전 필드 `isinstance(str)` 를 검사한다.)

필드	타입	내용
<code>question_id</code>	string	요청의 <code>question_id</code> 를 그대로 반환.
<code>question</code>	string	요청의 <code>question</code> 을 그대로 반환.
<code>retrieved_context</code>	string	답변 생성에 참고한 검색 문서 근거. 5-1 형식. 근거가 없으면 빈 문자열.
<code>think_trace</code>	string	사고·추론·도구 사용 과정을 담은 JSON 문자열 . 5-2 구조.
<code>answer</code>	string	최종 생성 답변. 근거 공시 표기를 포함한다. 답변 불가·근거 부족이면 결론 문장 + 이유 + (필요 시) 재질문 안내. 공개 문장은 약 300자로 제한.

4-1. 응답 본문 예시

```

{
  "question_id": "Q-001",
  "question": "삼성전자의 2024년 연결기준 영업이익은 얼마인가?",
  "retrieved_context": "[출처: 삼성전자 2024 사업보고서 | 재무제표 | 연결 손익계산서][문서ID: periodic_20250311001085::chunk_00042]\n영업이익 ... 32,725,961(백만원)",
  "think_trace": "{\"intent\":{\"status\":\"ok\"},\"retriever_result\":{\"status\":\"ok\"},\"reasoner_result\":{\"status\":\"success\"},\"validator_result\":{\"status\":\"success\"},\"supervisor\":{\"phase\":\"after_validator\",\"action\":\"finish\"},\"reason\":\"Validator 처리가 완료되었습니다.\",\"search_attempts\":0,\"planner_attempts\":0,\"regeneration_attempts\":0,\"validation_attempts\":1,\"termination

```

```

_reason\":null}}\",
  \"answer\": \"삼성전자의 2024년 연결기준 영업이익은 32조 7,259억원
입니다. (근거: 삼성전자 2024 사업보고서 연결 손익계산서)\"
}

```

5-1. `retrieved_context` 형식

`reasoner_result.citations` 목록을 다음 형태로 이어 붙인 것이다. 항목 사이는 빈 줄(`\n\n`)로 구분한다.

```

[출처: {source}][문서ID: {document_id}]
{evidence}

```

- `source` — 사람이 읽는 출처 표기(기업 · 보고서 · 섹션).
- `document_id` — 내부 문서/청크 식별자.
- `evidence` — 근거 본문. 없으면 `text` 값을 사용한다.
- `citations` 가 리스트가 아니거나 비어 있으면 빈 문자열.

5-2. `think_trace` 구조

`think_trace` 는 문자열이지만 내용은 JSON 객체다(`json.loads` 로 다시 파싱 가능).
`integration/api.py::_think_trace()` 가 만들고 `_redact_trace()` 가 다듬는다.

```

{
  "intent": { "status": ..., "warnings": [...], "trace": ..., "provider_status": ..., "failure_reason_code": ... },
  "retriever_result": { "status": ..., "warnings": [...], "trace": ..., "subqueries": [ { "subquery_id": ..., "status": ... } ] },
  "reasoner_result": { "status": ..., "warnings": [...], "trace": ..., "subqueries": [...] },
  "validator_result": { "status": ..., "warnings": [...], "trace": ... },
  "interpreter_think_trace": "Interpreter 규칙 추적 요약 문자열",
  "analysis_plan": {
    "status": ..., "failure_reason": ..., "trace": [...],

```

```

    "requirements": 0, "steps": 0
  },
  "supervisor": {
    "phase": "after_interpreter | after_retriever | after_r
easoner | after_validator",
    "action": "run_retriever | retry_search | run_calculati
on_planner | run_reasoner | run_validator | request_clarifi
cation | unanswerable | fail_closed | regenerate_answer | f
inish",
    "reason": "선택 이유 문자열",
    "search_attempts": 0,
    "planner_attempts": 0,
    "regeneration_attempts": 0,
    "validation_attempts": 0,
    "termination_reason": null
  }
}

```

`interpreter_think_trace` 와 `analysis_plan` 은 해당 정보가 있을 때만 포함된다.

`intent` ~ `validator_result` 하위 필드는 원본에 존재하는 키만 포함된다.

마스킹 규칙(`_redact_trace`):

- 다음 키는 어느 위치에서든 제거한다: `api_key`, `authorization`, `prompt`, `messages`, `content`, `question`.
- 문자열은 앞 500자까지만 남긴다.
- 리스트는 앞 32개 항목까지만 남긴다.
- 중첩 깊이가 4를 넘으면 `"[truncated]"` 로 자른다.
- API key, 원시 provider 오류 메시지, 검색 점수는 포함하지 않는다.

6. 오류 응답

FastAPI 표준에 따라 오류 본문은 `{"detail": ...}` 형태다. `detail` 은 문자열 또는 객체다.

코드	상황	<code>detail</code>	추가 헤더
422 Unprocessable Entity	<code>question_id</code> 또 는 <code>question</code> 누락 ·형식 오류	FastAPI 검증 오류 배열 (4-1)	없음

코드	상황	detail	추가 헤더
429 Too Many Requests	CLOVA provider 용량 한도	{ "code": "PROVIDER_RATE_LIMITED", "error_type": "...", "retry_after_seconds": <float> }	reset 시간을 알면 Retry-After: < 초>
503 Service Unavailable	파이프라인 미준비 (초기화 실패)	"pipeline is not ready" (문 자열)	없음
503 Service Unavailable	실행 중 예기치 못한 파이프라인 오류	{ "code": "PIPELINE_ERROR", "error_type": "<예외 클래스명 >" }	없음

- provider 메시지·프롬프트·API key는 서버 로그에만 남기고 응답에는 넣지 않는다.
- `answer` 자체가 근거 부족으로 차단될 때는 오류 코드가 아니라 200 으로 반환하며,
`answer` 문자열에 안전한 결론 문장과 이유를 담는다. 내부 `failure_reason_code` 는
`think_trace` 에만 기록한다.

6-1. 422 본문 예시 (question 누락)

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["query", "question"],
      "msg": "Field required",
      "input": null
    }
  ]
}
```

7. GET /ready 와 GET /health

GET /ready

파이프라인을 지연 초기화하여 Interpreter corpus, Retriever 저장소, provider 설정을
포함한
실행 준비 상태를 확인한다.

- 준비됨: 200 OK

```
{ "status": "ready", "implementation": "four-stage" }
```

- 준비 안 됨: 503 Service Unavailable

```
{ "detail": "pipeline is not ready" }
```

GET /health

FastAPI 프로세스 liveness만 확인한다. corpus나 DB가 마운트되지 않아도 200 을 반환한다.

```
{ "status": "ok", "implementation": "four-stage" }
```

8. JSON Schema

8-1. 요청 (GET /answer 쿼리 파라미터)

```
{
  "$schema": "<https://json-schema.org/draft/2020-12/schema>",
  "title": "AnswerRequestQuery",
  "type": "object",
  "properties": {
    "question_id": { "type": "string", "minLength": 1 },
    "question": { "type": "string", "minLength": 1 }
  },
  "required": ["question_id", "question"],
  "additionalProperties": false
}
```

8-2. 응답 (200 OK 본문)

```
{
  "$schema": "<https://json-schema.org/draft/2020-12/schema>",
  "title": "SubmissionResponse",

```

```

    "type": "object",
    "properties": {
      "question_id": { "type": "string" },
      "question": { "type": "string" },
      "retrieved_context": { "type": "string" },
      "think_trace": { "type": "string", "description":
"JSON 문자열. 파싱하면 8-3 구조." },
      "answer": { "type": "string" }
    },
    "required": ["question_id", "question", "retrieved_context", "think_trace", "answer"],
    "additionalProperties": false
  }

```

8-3. **think_trace** 파싱 후 구조

```

{
  "$schema": "<https://json-schema.org/draft/2020-12/schema>",
  "title": "ThinkTrace",
  "type": "object",
  "properties": {
    "intent": { "$ref": "#/$defs/stageTrace" },
    "retriever_result": { "$ref": "#/$defs/stageTrace" },
    "reasoner_result": { "$ref": "#/$defs/stageTrace" },
    "validator_result": { "$ref": "#/$defs/stageTrace" },
    "interpreter_think_trace": { "type": "string" },
    "analysis_plan": {
      "type": "object",
      "properties": {
        "status": { "type": ["string", "null"] },
        "failure_reason": { "type": ["string", "null"] },
        "trace": { "type": "array", "items": {} },
        "requirements": { "type": "integer", "minimum": 0
      },
      "steps": { "type": "integer", "minimum": 0
    },
  },
}

```

```

        "additionalProperties": false
    },
    "supervisor": {
        "type": "object",
        "properties": {
            "phase": { "type": ["string", "null"], "enum": ["after_interpreter", "after_retriever", "after_reasoner", "after_validator", null] },
            "action": { "type": ["string", "null"] },
            "reason": { "type": ["string", "null"] },
            "search_attempts": { "type": "integer", "minimum": 0 },
            "planner_attempts": { "type": "integer", "minimum": 0 },
            "regeneration_attempts": { "type": "integer", "minimum": 0 },
            "validation_attempts": { "type": "integer", "minimum": 0 },
            "termination_reason": { "type": ["string", "null"] }
        },
        "required": ["phase", "action", "reason"],
        "additionalProperties": false
    }
},
"required": ["supervisor"],
"additionalProperties": false,
"$defs": {
    "stageTrace": {
        "type": "object",
        "properties": {
            "status": { "type": ["string", "null"]
},
            "warnings": { "type": "array", "items":
{} },
            "trace": {},
            "provider_status": {},
            "failure_reason_code": { "type": ["string", "null"]

```

```

    },
    "subqueries": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "subquery_id": {},
          "status": { "type": ["string", "null"] }
        }
      }
    },
    "additionalProperties": false
  }
}

```

8-4. 오류 응답 (4xx / 5xx 본문)

```

{
  "$schema": "<https://json-schema.org/draft/2020-12/schema>",
  "title": "ErrorResponse",
  "type": "object",
  "properties": {
    "detail": {
      "oneOf": [
        { "type": "string" },
        {
          "type": "object",
          "properties": {
            "code": { "type": "string", "enum": ["PROVIDER_RATE_LIMITED", "PIPELINE_ERROR"] },
            "error_type": { "type": "string" },
            "retry_after_seconds": { "type": "number", "minimum": 0 }
          }
        }
      ],
      "required": ["code"]
    }
  }
}

```

```

    },
    {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "type": { "type": "string" },
          "loc": { "type": "array", "items": { "type":
["string", "integer"] } },
          "msg": { "type": "string" },
          "input": {}
        }
      }
    }
  ],
},
"required": ["detail"]
}

```

9. 동작·제약 메모

- **질문당 제한 시간:** 질문당 처리 예산은 300초다
(`integration/rate_limit.py::question_rate_limit_budget`). provider rate-limit 재시도 대기 상한(`CLOVA_RATE_LIMIT_MAX_WAIT`)은 코드 기본값 300초이나 배포 `.env` 는 60초로 낮춰 둔다. 클라이언트 타임아웃은 300초보다 크게 잡는다(예시 코드는 310초).
- **worker 1개:** `-workers 1` . 동시 요청은 순차 처리된다. 평가 시스템은 질문을 직렬로 보낸다고 가정한다.
- **지연 초기화:** 프로세스가 떠도 첫 `/ready` 또는 `/answer` 시점에 corpus·DB·provider 를 연다.
초기화 실패는 import 크래시가 아니라 HTTP `503` 으로 나타난다.
- **불변 필드:** `question_id` , `question` , `original_question` 은 실행 중 바뀌지 않는다.
검색 재시도는 내부 `search_query` 만 바꾼다.
- **반복 상한:** Supervisor 총 12단계, 검색 재시도 1회, 계산 계획 재시도 1회, 답변 재생성 1회,
그래프 `recursion_limit` 24.

- **CLOVA 챗 모델:** 답변 생성과 semantic validation에 쓰는 모델은 `CLOVA_CHAT_MODEL`로 정하며 현재 값은 `HCX-005` 다. `docker-compose.yml`의 `environment:` 블록에서 지정하고 `.env` 보다 우선한다.
- **CLOVA 비활성 시:** 운영 컨테이너는 `docker-compose.yml`에서 `CLOVA_LLM_ENABLED=true`를 고정하므로 평가 중에는 항상 활성화이다. 값을 끄면 Retriever는 `embedding_unavailable`을 반환하고 Validator는 semantic 검증을 통과할 수 없어 답변이 검증 실패로 처리된다. 형식만 달라지는 것이 아니라 답변이 차단된다.
- **근거 표시:** 모든 답변에 근거 공시를 표기한다(과제 필수 규칙).

10. 스모크 테스트

```
# 컨테이너 안에서
docker compose exec app python scripts/smoke_api.py --base-url <http://127.0.0.1:8000>

# 외부에서
curl -s "<http://49.50.142.35/answer?question_id=SMOKE-1&question=삼성전자의> 2024년 연결기준 영업이익은 얼마인가?" \
| python3 -m json.tool
```

`scripts/smoke_api.py`는 `/health` → `/ready` → `/answer` 순으로 확인하고, `/answer` 응답이 5개 문자열 필드 계약을 지키는지 검사한다.